

Vies e Variância

August 24, 2026

1 Erro em Modelos de Regressão

Para avaliarmos a qualidade de um modelo de regressão, precisamos de uma métrica que meça o quão bem ele se ajusta aos dados. Uma das métricas mais comuns para essa finalidade é o **Erro Quadrático Médio** (Mean Squared Error - MSE).

O MSE funciona como uma “função de custo” ou “função de perda”. Ele nos diz, em média, o quão longe as previsões do nosso modelo estão dos valores reais. O objetivo do treinamento de um modelo é minimizar essa função de custo.

A fórmula é a seguinte:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$$

Onde: * **n**: é o número total de observações (pontos de dados). * y_i : representa o valor real (a resposta correta) da i -ésima observação. * $\hat{f}(x_i)$: é o valor que o nosso modelo previu para a i -ésima observação. * $(y_i - \hat{f}(x_i))$: é o erro para uma única previsão.

Por que elevamos o erro ao quadrado? 1. **Eliminar Sinais Negativos**: Isso garante que erros, sejam eles positivos (previsão abaixo do real) ou negativos (previsão acima do real), não se anulem. 2. **Penalizar Erros Maiores**: Ao elevar ao quadrado, as diferenças maiores (previsões muito distantes do real) têm um impacto muito mais significativo no erro total do que as diferenças pequenas. Isso força o modelo a evitar erros grosseiros.

```
[1]: # Célula de configuração inicial.  
# Importamos a biblioteca para exibir imagens no notebook.  
# O comando '%matplotlib inline' garante que os gráficos gerados pelo Matplotlib  
# sejam exibidos diretamente na célula de saída do notebook.  
  
from IPython.display import Image  
%matplotlib inline
```

```
[2]: # Célula de configuração de estilo para os gráficos.  
# Aqui, personalizamos o tamanho da fonte para títulos, eixos e legendas  
# dos gráficos que serão gerados, tornando-os mais legíveis.  
  
import matplotlib.pyplot as plt
```

```
plt.rc('font', size=12)
plt.rc('axes', labelsz=14, titlesz=14)
plt.rc('legend', fontsize=12)
plt.rc('xtick', labelsz=10)
plt.rc('ytick', labelsz=10)
```

```
[3]: # Definição da semente aleatória (random seed).
# Isso garante que qualquer processo que utilize números aleatórios (como a
# divisão de dados)
# produza sempre os mesmos resultados, tornando nosso experimento reprodutível.

import numpy as np

np.random.seed(42)
```

```
[4]: # Exemplo Prático 1: Ajustando um Modelo de Regressão Linear

# Importação das bibliotecas necessárias
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

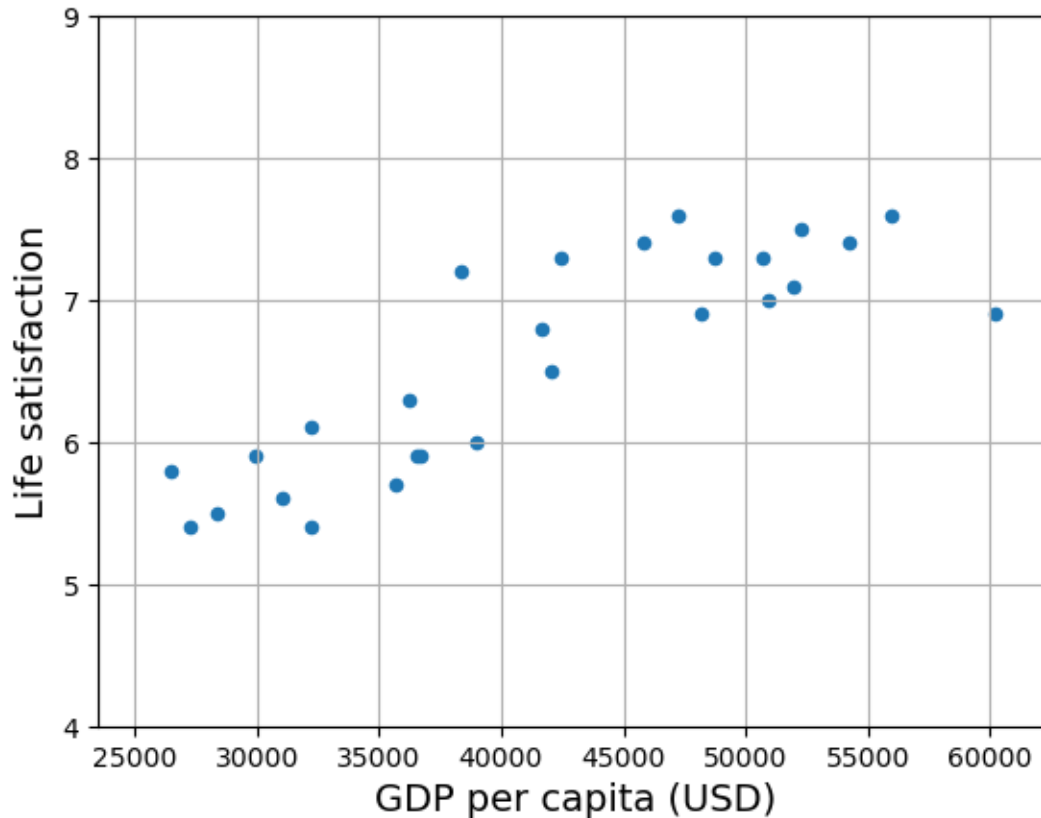
# 1. Carregando os dados: Usamos o pandas para ler um arquivo CSV que contém
# a relação entre o PIB per capita e o nível de satisfação com a vida em vários
# países.
lifesat = pd.read_csv("lifesat.csv")
X = lifesat[["GDP per capita (USD)"]].values # Característica (variável
# independente)
y = lifesat[["Life satisfaction"]].values # Alvo (variável dependente)

# 2. Visualizando os dados: Um gráfico de dispersão nos ajuda a entender a
# relação
# entre as variáveis. Podemos ver que há uma tendência linear positiva.
lifesat.plot(kind='scatter', grid=True,
             x="GDP per capita (USD)", y="Life satisfaction")
plt.axis([23_500, 62_500, 4, 9])
plt.show()

# 3. Escolha e Treinamento do Modelo:
# Instanciamos o modelo de Regressão Linear.
model = LinearRegression()
# O método .fit() "treina" o modelo, encontrando a melhor reta que se ajusta
# aos dados (X, y).
model.fit(X, y)

# 4. Fazendo uma Previsão:
```

```
# Usamos o modelo treinado para prever a satisfação com a vida para um novo
# valor de PIB per capita (neste caso, o de Chipre em 2020).
X_new = [[37_655.2]]
print(model.predict(X_new))
```



```
[[6.30165767]]
```

1.0.1 Substituindo o Modelo por um Mais Complexo

O modelo de regressão linear é um modelo **paramétrico** e **linear**, ou seja, ele assume que a relação entre as variáveis pode ser descrita por uma reta.

Agora, vamos experimentar um modelo **não-paramétrico** e **não-linear**: o *k-vizinhos mais próximos* (k-Nearest Neighbors - k-NN). Para um novo ponto de dados, este modelo encontra os 'k' pontos mais próximos no conjunto de treinamento e prevê a média de seus valores. Usaremos k=3.

```
[5]: # Exemplo Prático 2: Ajustando um Modelo k-NN

# 1. Importação do modelo
from sklearn.neighbors import KNeighborsRegressor
```

```

# 2. Escolha e Treinamento do Modelo:
# Instanciamos o modelo k-NN, especificando que ele deve usar 3 vizinhos
↳ (n_neighbors=3).
model = KNeighborsRegressor(n_neighbors=3)
# O método .fit() para o k-NN simplesmente armazena os dados de treinamento.
model.fit(X, y)

# 3. Fazendo uma Previsão:
# Fazemos a mesma previsão para Chipre. O resultado é a média dos níveis de
↳ satisfação
# dos 3 países com PIB per capita mais próximos ao de Chipre no nosso dataset.
X_new = [[37_655.2]] # PIB per capita de Chipre em 2020
print(model.predict(X_new))

```

```
[[6.33333333]]
```

```

[6]: # Preparação do ambiente para carregar mais dados, caso não existam localmente.
# Este código cria as pastas necessárias para organizar os datasets.

```

```

import urllib.request
from pathlib import Path

datapath = Path() / "datasets" / "lifesat"
datapath.mkdir(parents=True, exist_ok=True)

```

```

[7]: # Carregando os datasets da OCDE (Better Life Index) e do Banco Mundial (PIB
↳ per capita).
# Esses são os arquivos de origem usados para criar o 'lifesat.csv'.

```

```

oecd_bli = pd.read_csv(datapath / "oecd_bli.csv")
gdp_per_capita = pd.read_csv(datapath / "gdp_per_capita.csv")

```

```

[8]: # Pré-processamento e limpeza dos dados de PIB per capita.
# O objetivo é ter um DataFrame limpo com o país como índice e o PIB per capita
↳ de 2020 como valor.

```

```

gdp_year = 2020
gdppc_col = "GDP per capita (USD)"
lifesat_col = "Life satisfaction"

# Filtra o DataFrame para conter apenas os dados do ano de 2020.
gdp_per_capita = gdp_per_capita[gdp_per_capita["Year"] == gdp_year]
# Remove colunas desnecessárias.
gdp_per_capita = gdp_per_capita.drop(["Code", "Year"], axis=1)
# Renomeia as colunas para clareza.
gdp_per_capita.columns = ["Country", gdppc_col]
# Define a coluna 'Country' como o índice do DataFrame.

```

```
gdp_per_capita.set_index("Country", inplace=True)

# Exibe as 5 primeiras linhas do DataFrame processado para verificação.
gdp_per_capita.head()
```

[8]: GDP per capita (USD)

Country	
Afghanistan	1978.961579
Africa Eastern and Southern	3387.594670
Africa Western and Central	4003.158913
Albania	13295.410885
Algeria	10681.679297

[9]: # Definição dos limites para o eixo x do próximo gráfico.

```
min_gdp = 23_500
max_gdp = 62_500
```

[10]: # Este gráfico ilustra como diferentes parâmetros (e) resultam em
diferentes modelos de regressão linear.

```
lifesat.plot(kind='scatter', figsize=(5, 3), grid=True,
              x=gdppc_col, y=lifesat_col)

X = np.linspace(min_gdp, max_gdp, 1000)
min_life_sat = 4
max_life_sat = 9

# --- Modelo 1 (vermelho) ---
# Aqui, w1 ( ) = 4.2 e w2 ( ) = 0.
# A linha plotada é y = 4.2 + 0 * X, uma reta horizontal.
# Os textos refletem exatamente esses valores.
w1, w2 = 4.2, 0
plt.plot(X, w1 + w2 * 1e-5 * X, "r")
plt.text(40_000, 4.9, fr"$\theta_0 = {w1}$", color="r")
plt.text(40_000, 4.4, fr"$\theta_1 = {w2}$", color="r")

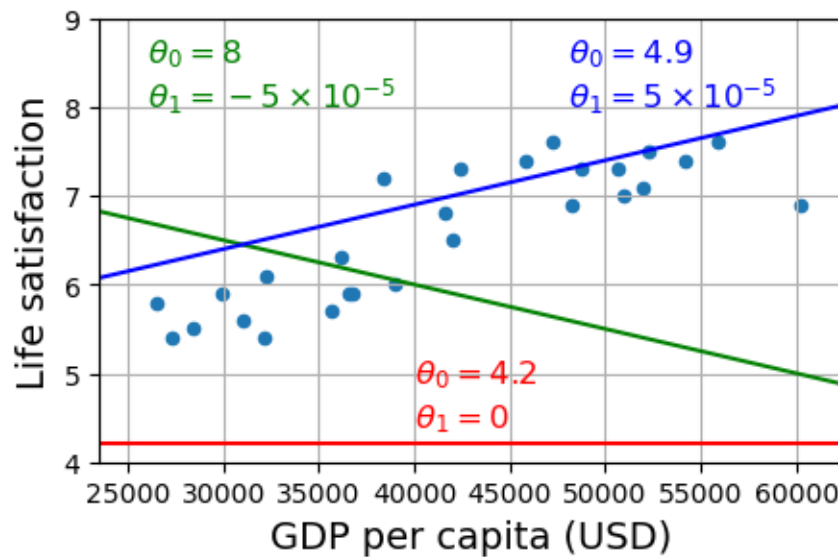
# --- Modelo 2 (verde) ---
# Aqui, w1 ( ) = 8 e w2 ( ) = -5.
# A linha plotada é y = 8 + (-5 * 1e-5) * X, uma reta com inclinação negativa.
# Os textos refletem exatamente esses valores.
w1, w2 = 8, -5
plt.plot(X, w1 + w2 * 1e-5 * X, "g")
plt.text(26_000, 8.5, fr"$\theta_0 = {w1}$", color="g")
plt.text(26_000, 8.0, fr"$\theta_1 = {w2} \times 10^{{-5}}$", color="g")

# --- Modelo 3 (azul) ---
```

```
# Aqui, w1 ( ) = 4.9 e w2 ( ) = 5.
# A linha plotada é  $y = 4.9 + (5 * 1e-5) * X$ , uma reta com inclinação positiva,
  ↳ que se ajusta bem aos dados.
# Os textos refletem exatamente esses valores.
w1, w2 = 4.9, 5
plt.plot(X, w1 + w2 * 1e-5 * X, "b")
plt.text(48_000, 8.5, fr"$\theta_0 = {w1}$", color="b")
plt.text(48_000, 8.0, fr"$\theta_1 = {w2} \times 10^{{-5}}$", color="b")

plt.axis([min_gdp, max_gdp, min_life_sat, max_life_sat])

plt.show()
```



1.0.2 Parâmetros do Modelo de Regressão Linear

Um modelo de regressão linear simples pode ser representado pela equação de uma reta:

$$\hat{y} = \theta_0 + \theta_1 x$$

Onde: * θ_0 (theta zero): é o **intercepto** (ou *bias term*), o valor de $\hat{y}$ quando x é zero. No gráfico, é onde a linha cruza o eixo Y. * θ_1 (theta um): é o **coeficiente angular** (ou peso), que representa a inclinação da reta. Ele nos diz o quanto $\hat{y}$ muda para cada unidade de mudança em x .

O algoritmo de regressão linear busca os “melhores” valores para θ_0 e θ_1 que minimizem o Erro Quadrático Médio (MSE), ou seja, que tornem a reta azul do gráfico anterior a que passa mais perto de todos os pontos de dados.

2 O Equilíbrio entre Viés e Variância (Bias-Variance Tradeoff)

O erro de um modelo de aprendizado de máquina pode ser decomposto em três componentes principais, que nos ajudam a diagnosticar problemas de performance:

- **Viés (Bias):** É o erro de **simplicidade**. Um modelo com alto viés faz suposições muito simplistas sobre os dados (ex: assumir que uma relação complexa é linear). Isso o impede de capturar a verdadeira relação entre as variáveis, levando ao **subajuste (underfitting)**. O modelo é consistentemente impreciso, mesmo com mais dados.
- **Variância (Variance):** É o erro de **complexidade excessiva**. Um modelo com alta variância é muito sensível às pequenas flutuações nos dados de treinamento. Ele “decora” os dados, incluindo o ruído, em vez de aprender a tendência geral. Como resultado, ele não generaliza bem para dados novos, levando ao **sobreajuste (overfitting)**.
- **Erro Irredutível (Ruído):** É o erro que não pode ser eliminado, não importa quão bom seja o modelo. Ele é uma propriedade inerente aos dados (ex: erros de medição, aleatoriedade natural).

A relação entre essas componentes pode ser expressa pela seguinte equação, que decompõe o Erro Quadrático Médio esperado:

$$E[(y - \hat{f}(x))^2] = \text{Var}(\hat{f}(x)) + [\text{Bias}(\hat{f}(x))]^2 + \text{Var}(\epsilon)$$

Onde: * y : é o valor real. * $\hat{f}(x)$: é a predição do modelo. * $E[\cdot]$: representa o valor esperado (a média sobre muitos conjuntos de dados de treinamento). * $[\text{Bias}(\hat{f}(x))]^2$: O **quadrado do viés** mede o quão longe, em média, as predições do modelo estão do valor real. * $\text{Var}(\hat{f}(x))$: A **variância** mede o quanto as predições para um mesmo ponto variam com diferentes conjuntos de dados de treinamento. * $\text{Var}(\epsilon)$: É a variância do erro irredutível (ruído).

O **tradeoff viés-variância** é o conceito central de que existe um equilíbrio delicado: * Modelos **simples** (baixa complexidade) têm **alto viés** e **baixa variância**. * Modelos **complexos** (alta complexidade) têm **baixo viés** e **alta variância**.

Nosso objetivo como cientistas de dados é encontrar um modelo com a complexidade certa, que minimize a soma do viés e da variância.

2.0.1 Definições Matemáticas

Viés: A diferença entre a predição média do modelo e o valor verdadeiro.

$$\text{Bias} = E[\hat{f}(x)] - f(x)$$

Variância: A variância das predições do modelo em torno de sua predição média.

$$\text{Var}(\hat{f}(x)) = E[(E[\hat{f}(x)] - \hat{f}(x))^2]$$

Decomposição do MSE: O Erro Quadrático Médio (MSE) é a soma da variância e do quadrado do viés (ignorando o erro irredutível, que é constante).

$$\text{MSE} = \text{Variância} + \text{Viés}^2$$

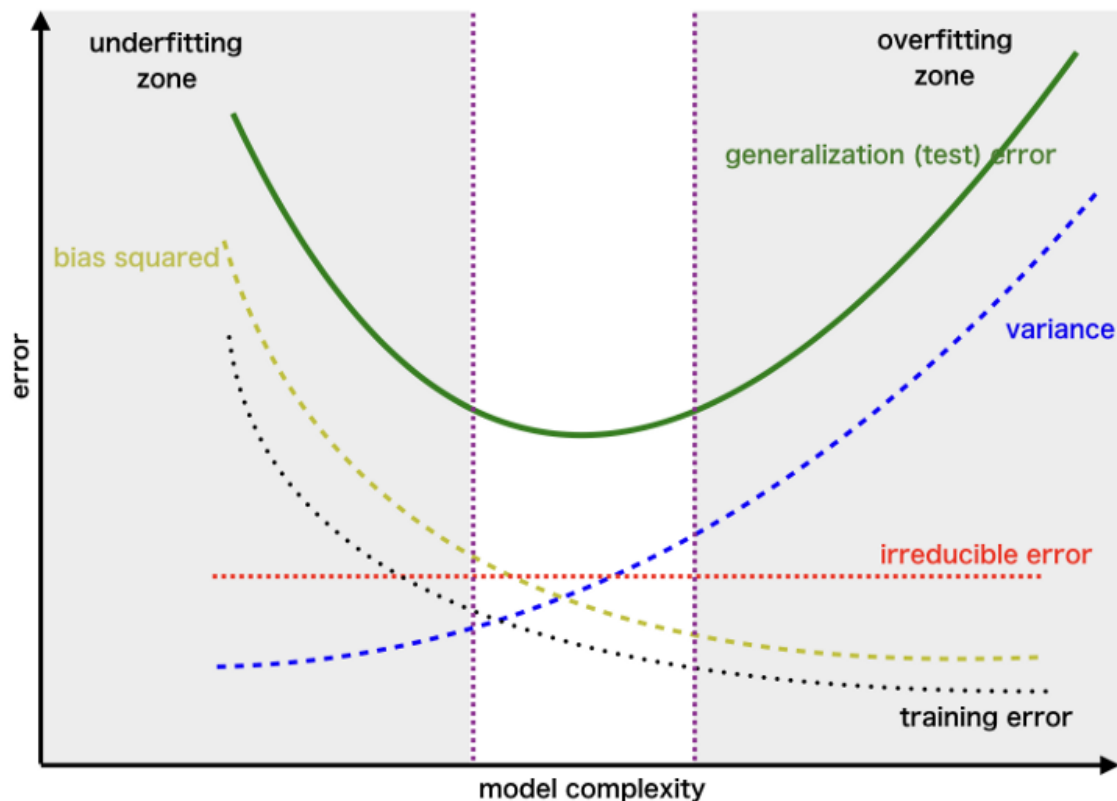
Usando a notação de previsão ($\hat{y}$) e valor real (y):

$$E[(\hat{y} - y)^2] = E[(\hat{y} - E[\hat{y}])^2] + (E[\hat{y}] - y)^2$$

```
[11]: # Este gráfico clássico ilustra visualmente o tradeoff viés-variância.
# - Eixo X: Complexidade do Modelo (de simples, como regressão linear, a
#       ↪ complexo, como uma rede neural profunda).
# - Eixo Y: Erro.
# - Viés (Bias): Diminui à medida que o modelo se torna mais complexo e
#       ↪ flexível.
# - Variância (Variance): Aumenta à medida que o modelo se torna mais complexo
#       ↪ e sensível ao ruído.
# - Erro Total: A soma do viés, variância e erro irreduzível. A curva em forma
#       ↪ de "U" mostra que o erro total
#       é mínimo em um ponto de complexidade intermediário, que é o nosso objetivo.

from IPython.display import Image
Image(filename='1_o00KYF7Z84nePqfsJ9E0WQ.png')
```

[11]:



2.0.2 A Decomposição na Prática

A complexidade de um modelo impacta diretamente o viés e a variância:

- **Baixa Complexidade:** Modelos mais simples (como a regressão linear) tendem a ter estimativas semelhantes para diferentes conjuntos de dados de treinamento. Isso significa que eles são estáveis, resultando em **baixa variância**. No entanto, sua simplicidade pode não ser suficiente para capturar a verdadeira relação nos dados (subajuste), levando a um **alto viés**.
- **Alta Complexidade:** Modelos mais complexos (como polinômios de grau elevado ou árvores de decisão profundas) podem se ajustar muito bem aos dados de treinamento, resultando em **baixo viés**. Contudo, eles são muito sensíveis às particularidades de cada conjunto de dados (sobreajuste), o que leva a uma **alta variância**.

O objetivo em aprendizado de máquina é encontrar um nível de complexidade que minimize o erro total, ou seja, que encontre o “ponto ideal” no fundo da curva “U” do erro total.

O processo a seguir implementa a decomposição usando uma técnica comum chamada **bootstraping**. A ideia é simular o treinamento do modelo em diferentes conjuntos de dados, criando várias “versões” do nosso modelo. Faremos o seguinte:

1. **Amostragem (Bootstrap):** Em um loop, vamos criar múltiplas amostras do nosso conjunto de treinamento, selecionando dados aleatoriamente *com reposição*.
2. **Treinamento:** Para cada amostra, treinaremos um novo modelo de Regressão Linear.
3. **Previsão:** Cada um desses modelos fará previsões no mesmo conjunto de teste.
4. **Cálculo:** Com todas as previsões para o conjunto de teste, podemos calcular:
 - **Viés²:** A diferença ao quadrado entre a média de todas as previsões e o valor real.
 - **Variância:** A variância das previsões entre si.
 - **MSE:** O erro quadrático médio total.

Este método nos permite estimar os componentes do erro sem depender de uma biblioteca externa.

```
[12]: # Estimando o viés e a variância para um modelo de regressão linear
      ↪ (implementação manual com bootstrapping)

# Importação de bibliotecas necessárias
import numpy as np
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Carregando o conjunto de dados "housing"
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.csv'
dataframe = read_csv(url, header=None)
data = dataframe.values
X, y = data[:, :-1], data[:, -1]

# Dividindo os dados em conjuntos de treinamento e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
      ↪ random_state=1)
```

```

# Definindo o modelo de Regressão Linear
model = LinearRegression()

# Parâmetros para o bootstrap
n_rounds = 200
np.random.seed(1)

# Array para armazenar as previsões de cada rodada
all_predictions = np.zeros((n_rounds, len(y_test)))

# Loop de bootstrap
for i in range(n_rounds):
    # Amostragem com reposição do conjunto de treinamento
    train_indices = np.random.choice(len(X_train), len(X_train), replace=True)
    X_train_boot = X_train[train_indices]
    y_train_boot = y_train[train_indices]

    # Treina o modelo na amostra bootstrap
    model.fit(X_train_boot, y_train_boot)

    # Faz e armazena as previsões no conjunto de teste original
    predictions = model.predict(X_test)
    all_predictions[i, :] = predictions

# Calculando as componentes do erro
# MSE: Erro médio de todas as previsões em relação aos valores reais
mse = np.mean((all_predictions - y_test)**2)

# Bias2: (Média das previsões - Valor real)2
main_predictions = np.mean(all_predictions, axis=0)
bias_sq = np.mean((main_predictions - y_test)**2)

# Variância: Variância das previsões
variance = np.mean(np.var(all_predictions, axis=0))

# Exibindo os resultados
print('MSE: %.3f' % mse)
print('Bias: %.3f' % bias_sq)
print('Variance: %.3f' % variance)

```

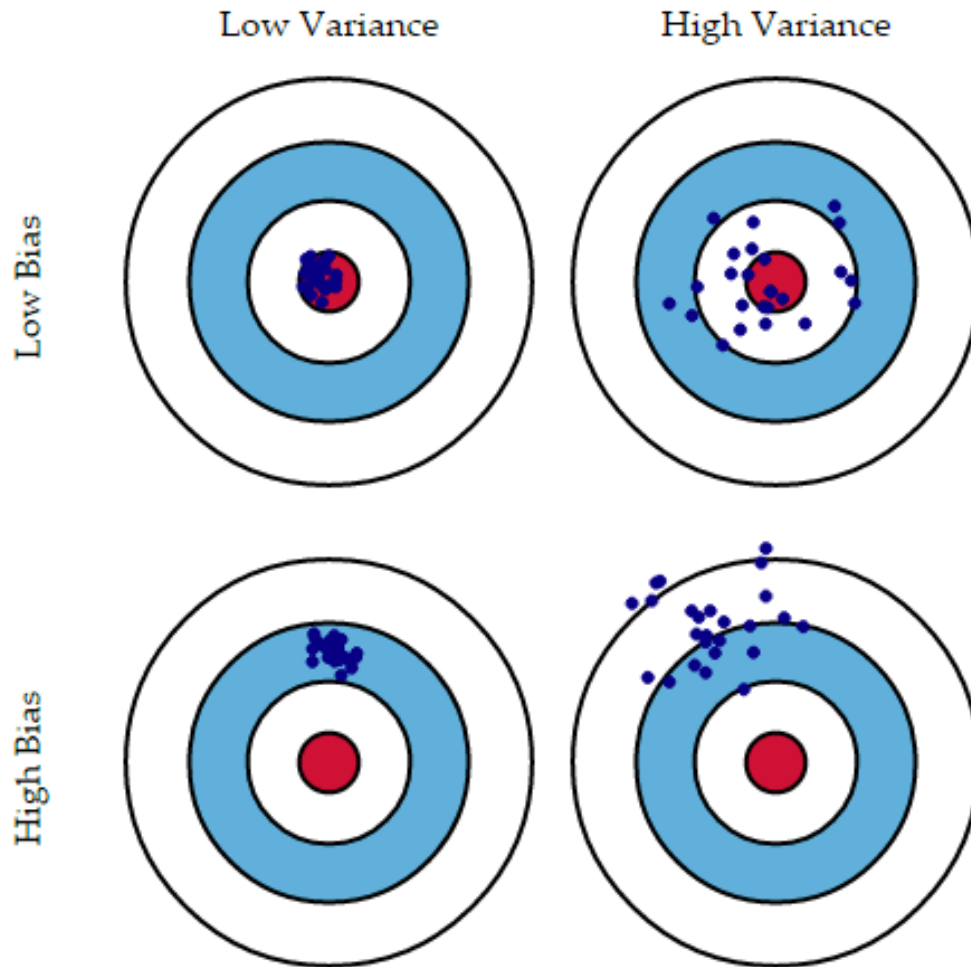
```

MSE: 22.418
Bias: 20.744
Variance: 1.674

```

```
[13]: Image(filename='Variancia_Vies.png')
```

```
[13]:
```



2.1 Tradeoff Viés-Variância na Prática: Um Exemplo com Regressão Polinomial

Para ilustrar o tradeoff de forma mais visual, vamos criar um conjunto de dados sintético onde conhecemos a “verdade”. Vamos supor que a verdadeira função que gera os dados seja:

$$f(x) = \frac{1}{2}x + \sqrt{\max(x, 0)} - \cos x + 2$$

E o ruído nos dados seja modelado por uma distribuição Gaussiana com média zero e desvio padrão 1:

$$\epsilon \sim N(0, 1)$$

Assim, os dados que observamos (nossos pontos y) são gerados pela equação:

$$y = f(x) + \epsilon$$

Vamos gerar 1000 pontos a partir desse processo para visualizar a relação.

```
[14]: # Gerando e visualizando os dados sintéticos

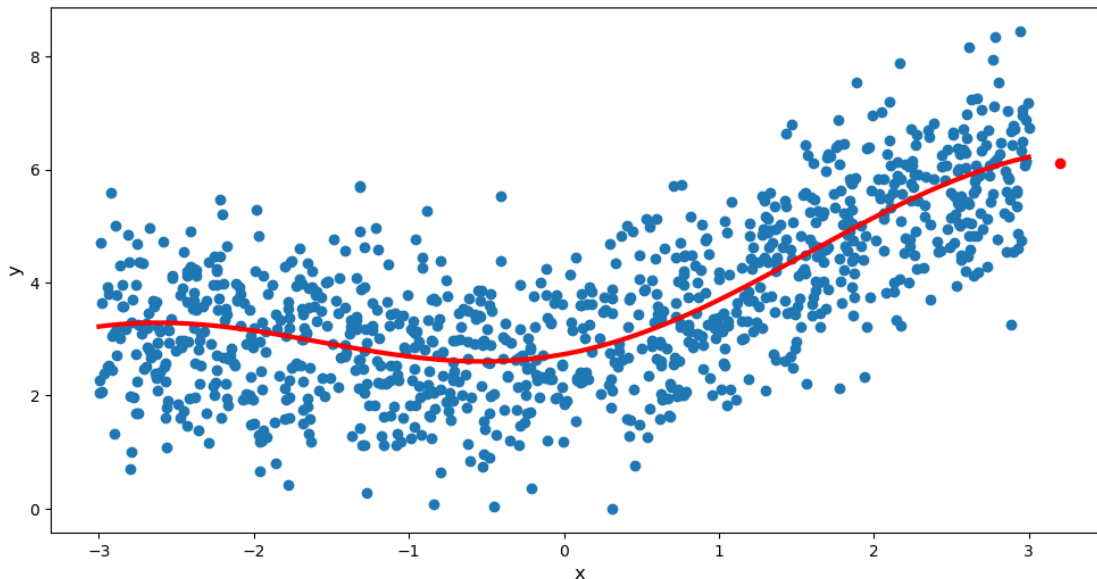
import numpy as np
import scipy.stats as stats
import matplotlib.pyplot as plt
%matplotlib inline

# Definição da função verdadeira f(x)
def f(x):
    return .5 * x + np.sqrt(np.max(x, 0)) - np.cos(x) + 2

# Geração dos dados com ruído
N = 1000
sigma_epsilon = 1
x_max = 3
x_test = 3.2
x = x_max * (2 * np.random.rand(N) - 1)
epsilon = sigma_epsilon * np.random.randn(N)

y = f(x) + epsilon
y_test = f(x_test) + sigma_epsilon * np.random.randn()

# Plotando os dados (pontos azuis), a função verdadeira (linha vermelha)
# e um ponto de teste (ponto vermelho) que usaremos para avaliação.
plt.figure(figsize=(12, 6))
x_range = np.linspace(-x_max, x_max, 1000)
plt.scatter(x, y)
plt.plot(x_range, f(x_range), 'r', linewidth=3.0)
plt.scatter(x_test, y_test, c='r')
plt.xlabel('x', size=12)
plt.ylabel('y', size=12)
plt.xticks(np.arange(-x_max, x_max + 1))
plt.show()
```



Os pontos azuis representam os pares (x, y) observados, e a linha vermelha é a função verdadeira $f(x)$ que queremos que nosso modelo aprenda. O ponto em vermelho é um ponto de teste que usaremos para avaliar as previsões dos nossos modelos.

Note que $f(x)$ tem um comportamento claramente não-linear, o que torna a modelagem com modelos lineares simples bastante desafiadora.

2.1.1 Modelando com Regressões Polinomiais

Vamos agora ajustar regressões polinomiais de diferentes graus (que representam diferentes níveis de complexidade) a esses dados. A função de uma regressão polinomial é:

$$\hat{f}(x) = w_0 + w_1x + w_2x^2 + \dots + w_dx^d$$

onde d é o grau do polinômio, que controla a complexidade do modelo.

Para cada grau de complexidade (1, 2, 3 e 5), vamos repetir o seguinte processo 6 vezes: 1. Selecionar aleatoriamente 20 pontos do nosso conjunto de dados total. 2. Treinar uma regressão polinomial nesses 20 pontos. 3. Visualizar a função ajustada.

Isso nos permitirá observar como a **variância** (a mudança no modelo ajustado com base nos dados de treinamento) aumenta com a complexidade do modelo.

```
[15]: # Função para ajustar e prever com um modelo polinomial
def f_hat(x, w):
    d = len(w) - 1
    # Esta linha calcula o valor do polinômio para um dado x e coeficientes w
    return np.sum(w * np.power(x, np.expand_dims(np.arange(d, -1, -1), 1)).T, 1)
```

```

# Parâmetros para os experimentos
n = int(.02 * N) # Usaremos 2% dos dados (20 pontos) para cada treinamento
x_test = 3.2
x_range = np.linspace(-x_max, x_max, 1000)
colors = np.array(['tab:green', 'tab:purple', 'tab:cyan', 'tab:orange'])
d_arr = [1, 2, 3, 5] # Graus dos polinômios que vamos testar

# Realizando 6 experimentos com diferentes subconjuntos de dados
cnt = 1
fig, axs = plt.subplots(2, 3, sharey=True, figsize=(15, 9))
for i in range(2):
    for j in range(3):
        # Seleciona um subconjunto aleatório de dados para este experimento
        idx = np.random.permutation(N)[:n]
        x_train, y_train = x[idx], y[idx]

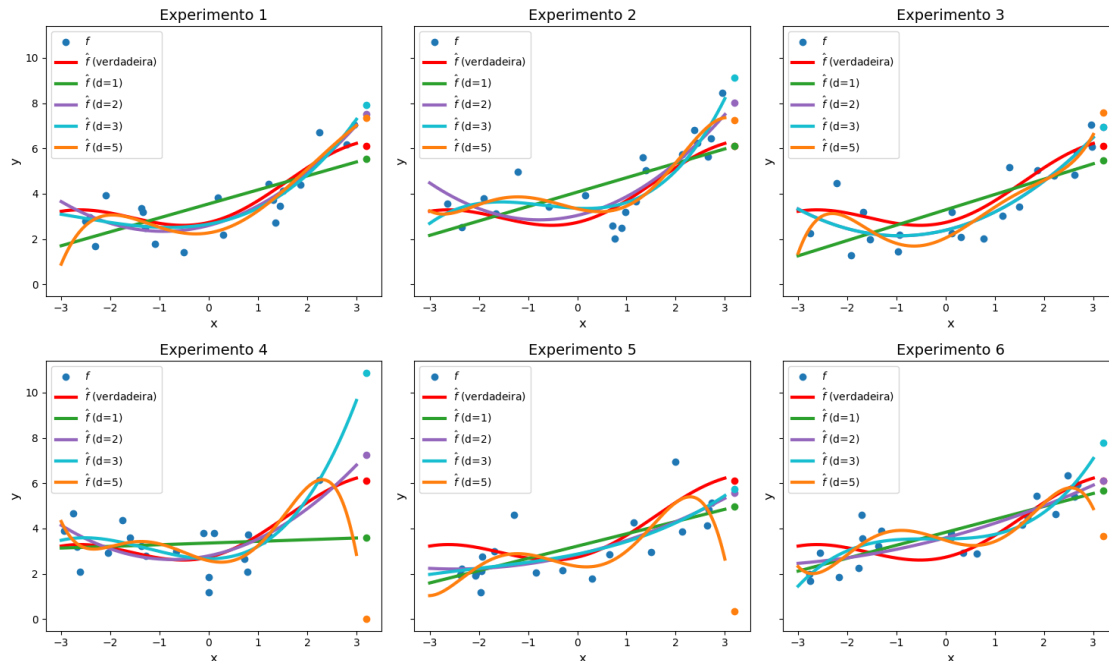
        # Ajusta modelos de diferentes graus
        w = []
        for d in d_arr:
            w.append(np.polyfit(x_train, y_train, d))

        # Plota os dados de treinamento, a função verdadeira e as funções
        ↪ajustadas
        axs[i, j].scatter(x_train, y_train)
        axs[i, j].plot(x_range, f(x_range), 'r', linewidth=3.0)
        for k in range(len(w)):
            axs[i, j].plot(x_range, f_hat(x_range, w[k]), colors[k],
            ↪linewidth=3.0)

        # Plota as previsões para o ponto de teste
        axs[i, j].scatter(x_test, y_test, c='r')
        for k in range(len(w)):
            axs[i, j].scatter(x_test, f_hat(x_test, w[k]), c=colors[k])

        axs[i, j].set_xlabel('x', size=12)
        axs[i, j].set_ylabel('y', size=12)
        axs[i, j].legend([r'$f$', r'$\hat{f}$ (verdadeira)', r'$\hat{f}$ (d=1)',
            r'$\hat{f}$ (d=2)', r'$\hat{f}$ (d=3)', r'$\hat{f}$
            ↪(d=5)'], fontsize=10)
        axs[i, j].title.set_text('Experimento {}'.format(cnt))
        cnt += 1
plt.tight_layout()
plt.show()

```



Observações dos Gráficos:

- **Pontos Azuis:** Os 20 pontos de treinamento aleatórios para cada um dos 6 experimentos.
- **Linha Vermelha:** A função verdadeira $f(x)$ que estamos tentando aprender.
- **Linhas Coloridas:** Os modelos polinomiais ajustados.

É nítido que os modelos mais simples ($d=1$, linha verde) variam muito pouco entre os diferentes experimentos. Eles são estáveis, ou seja, têm **baixa variância**. Por outro lado, os modelos mais complexos ($d=5$, linha laranja) mudam drasticamente de forma a cada novo conjunto de 20 pontos. Eles são instáveis e têm **alta variância**.

Este é o **problema da variância**: um modelo mais simples é mais robusto (menos sensível) a mudanças nos dados de treinamento.

No entanto, também podemos ver o **problema do viés**: os modelos de $d=1$ (verde) estão, em média, muito distantes da curva vermelha verdadeira. Eles são muito simples para capturar a não-linearidade dos dados, resultando em **alto viés**.

A seguir, vamos quantificar essa observação simulando 10.000 experimentos para analisar a distribuição das previsões no nosso ponto de teste.

[16]: *# Simulação em larga escala para analisar a distribuição das previsões no ponto de teste.*

```
R = 10000 # Número de experimentos
d_arr = [1, 2, 3, 5]
y_hat_test = np.zeros((len(d_arr), R))
```

```

# Repete o processo de amostragem, treinamento e predição R vezes
for r in range(R):
    n = int(.02 * N)
    idx = np.random.permutation(N)[:n]
    x_train, y_train = x[idx], y[idx]

    for k in range(len(d_arr)):
        d = d_arr[k]
        w = np.polyfit(x_train, y_train, d)
        y_hat_test[k, r] = f_hat(x_test, w)

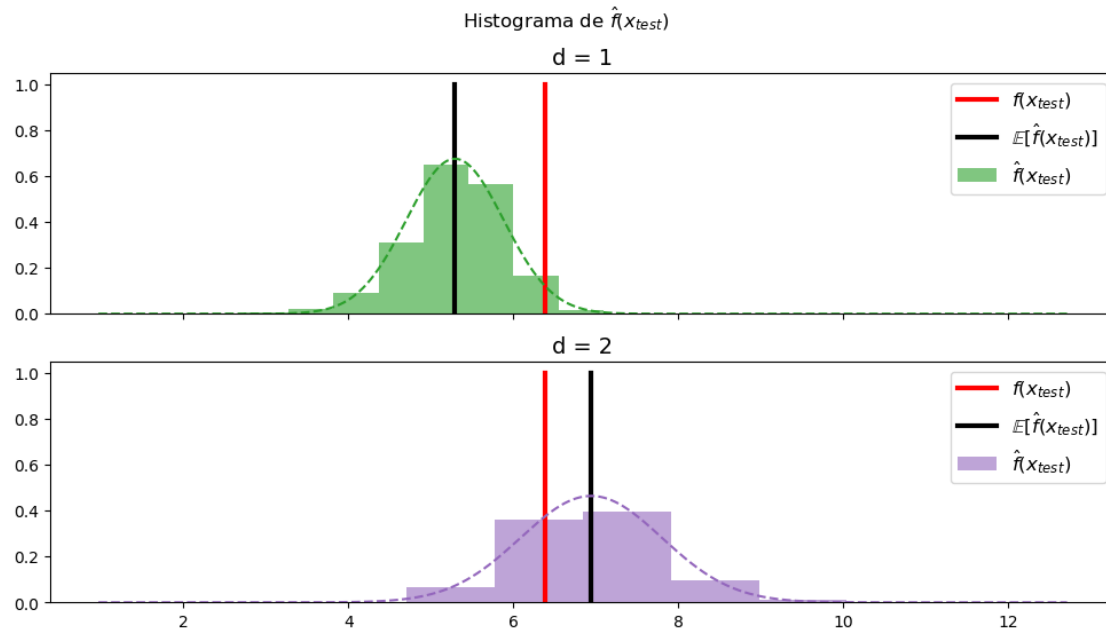
# Calcula a média e o desvio padrão das R predições para cada grau de polinômio
y_hat_test_mean = np.mean(y_hat_test, 1)
y_hat_test_std = np.std(y_hat_test, 1)

# Plotando os histogramas das predições para d=1 e d=2
fig, axs = plt.subplots(2, 1, sharex=True, sharey=True, figsize=(12, 6))
for k in range(2):
    # O histograma mostra a distribuição das 10.000 previsões para o ponto de
    ↪ teste
    axs[k].hist(y_hat_test[k], density=True, color=colors[k], alpha=0.6)
    xlim = axs[k].get_xlim()
    # Linha vermelha: o valor verdadeiro da função no ponto de teste
    axs[k].plot([f(x_test), f(x_test)], [0, 1], 'r', linewidth=3.0)
    # Linha preta: a média de todas as previsões (o "valor esperado" da
    ↪ predição)
    axs[k].plot([y_hat_test_mean[k], y_hat_test_mean[k]], [0, 1], c='k',
    ↪ linewidth=3.0)
    axs[k].title.set_text('d = {}'.format(d_arr[k]))
    axs[k].legend([r'$f(x_{test})$', r'$\mathbb{E}[\hat{f}(x_{test})]$',
    ↪ r'$\hat{f}(x_{test})$'], fontsize=12)

# Adicionando a curva da distribuição normal ajustada para visualizar a
    ↪ dispersão
for k in range(2):
    x_range = np.linspace(xlim[0], xlim[1], 1000)
    axs[k].plot(x_range, stats.norm.pdf(x_range, y_hat_test_mean[k],
    ↪ y_hat_test_std[k]), color=colors[k], ls='--')

plt.suptitle(r'Histograma de $\hat{f}(x_{test})$', size=12)
plt.show()

```



Análise dos Histogramas:

- **Gráfico para d=1 (Acima):**
 - **Viés:** A linha preta (média das previsões) está visivelmente longe da linha vermelha (valor real). A diferença entre elas é o **viés**. Vemos, portanto, um **viés alto**.
 - **Variância:** O histograma é estreito e concentrado. Isso significa que as previsões não mudaram muito entre os 10.000 experimentos. Temos, portanto, uma **variância baixa**.
- **Gráfico para d=2 (Abaixo):**
 - **Viés:** A linha preta agora está muito mais próxima da linha vermelha. O **viés diminuiu** significativamente.
 - **Variância:** O histograma está mais “espalhado”, mais largo. Isso indica que as previsões variaram mais entre os experimentos, ou seja, a **variância aumentou**.

Essa comparação ilustra perfeitamente o **tradeoff**: ao aumentar a complexidade do modelo (de d=1 para d=2), diminuímos o viés, mas ao custo de aumentar a variância.

[18]: *# Gráfico final que decompõe o erro de teste em suas componentes:
Viés ao quadrado, Variância e Erro de Ruído.*

```
R = 10000
n = int(.02 * N)
n_test = 1000
d_arr = np.arange(5) # Agora testamos os graus de 0 a 4

# Geração de um conjunto de teste fixo para avaliação
x_test = x_max + np.random.rand(n_test) - .5
epsilon = sigma_epsilon * np.random.randn(n_test)
```

```

y_test = f(x_test) + epsilon

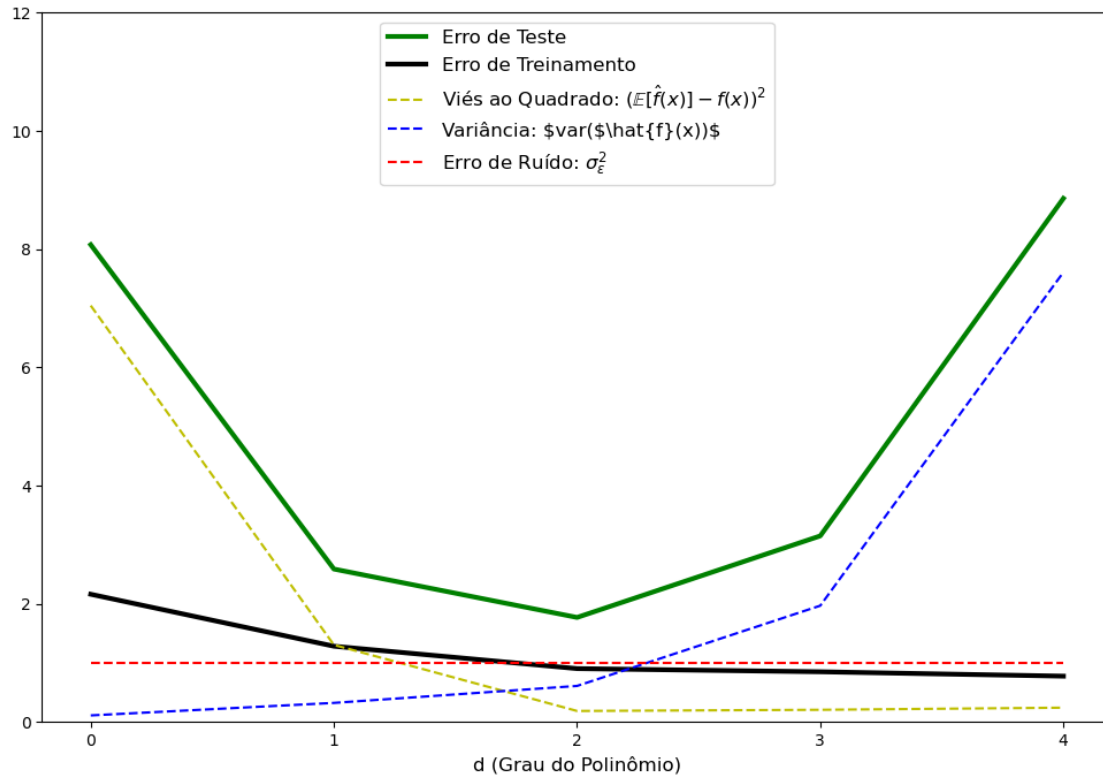
# Arrays para armazenar os resultados dos 10.000 experimentos
train_squared_error = np.zeros((len(d_arr), R))
y_hat_test = np.zeros((len(d_arr), R, n_test))

# Simulação principal
for r in range(R):
    n = int(.02 * N)
    idx = np.random.permutation(N)[:n]
    x_train, y_train = x[idx], y[idx]
    for k in range(len(d_arr)):
        d = d_arr[k]
        w = np.polyfit(x_train, y_train, d)
        # Erro no conjunto de treinamento para esta simulação
        train_squared_error[k, r] = np.mean((y_train - f_hat(x_train, w)) ** 2)
        # Previsões no conjunto de teste para esta simulação
        y_hat_test[k, r, :] = f_hat(x_test, w)

# Cálculo das componentes do erro, fazendo a média sobre os 10.000 experimentos
test_squared_error = np.mean((y_hat_test - y_test) ** 2, 1) # Erro de teste
↳ médio
bias_squared = (np.mean(y_hat_test, 1) - f(x_test)) ** 2 # Viés ao quadrado
↳ médio
var_y_hat_test = np.var(y_hat_test, 1) # Variância média

# Plotando os resultados
plt.figure(figsize=(12, 8))
plt.plot(d_arr, np.mean(test_squared_error, 1), 'g', linewidth=3.0)
plt.plot(d_arr, np.mean(train_squared_error, 1), 'k', linewidth=3.0)
plt.plot(d_arr, np.mean(bias_squared, 1), 'y--')
plt.plot(d_arr, np.mean(var_y_hat_test, 1), 'b--')
plt.plot(d_arr, (sigma_epsilon ** 2) * np.ones_like(d_arr), 'r--')
plt.xticks(d_arr)
plt.xlabel('d (Grau do Polinômio)', size=12)
plt.legend(['Erro de Teste', 'Erro de Treinamento', r'Viés ao Quadrado:
↳  $\mathbb{E}[\hat{f}(x) - f(x)]^2$ ',
           r'Variância:  $\text{var}(\hat{f}(x))$ ', r'Erro de Ruído:
↳  $\sigma_{\epsilon}^2$ '], loc='upper center', fontsize=12)
plt.ylim([0, 12])
plt.show()

```



Conclusão da Análise Gráfica Final:

Este gráfico resume todo o conceito do tradeoff viés-variância.

- **Linha Preta (Erro de Treinamento):** O erro nos dados de treinamento diminui consistentemente à medida que a complexidade (grau d) do modelo aumenta. Isso é esperado, pois modelos mais complexos conseguem “memorizar” melhor os dados de treinamento, levando ao sobreajuste.
- **Linha Amarela (Viés ao Quadrado):** O viés também diminui com o aumento da complexidade. Modelos mais flexíveis são capazes de se aproximar melhor da forma da função verdadeira.
- **Linha Azul (Variância):** A variância, por outro lado, aumenta com a complexidade. Modelos mais flexíveis são mais instáveis e sensíveis ao ruído dos dados de treinamento.
- **Linha Vermelha (Erro de Ruído):** É constante, pois é uma característica intrínseca dos dados e não depende do modelo.
- **Linha Verde (Erro de Teste):** Esta é a curva mais importante, pois nos diz como o modelo se comporta em dados novos. Ela é a **soma do viés ao quadrado, da variância e do erro de ruído**. Observe seu formato de “U”:
 - Para **modelos simples (baixo d)**, o erro de teste é alto devido ao **alto viés**.
 - Para **modelos complexos (alto d)**, o erro de teste é alto devido à **alta variância**.

- O ponto mínimo da curva verde ocorre em **$d=2$** . Este é o “ponto ideal”, o modelo que melhor generaliza para novos dados, pois alcança o equilíbrio ótimo entre viés e variância.

[]: